NOTE Communicated by Leo Breiman

N-tuple Network, CART, and Bagging

Aleksander Kołcz
Electrical and Computer Engineering Department, University of Colorado at Colorado Springs, Colorado Springs, CO 80918, U.S.A.

Similarities between bootstrap aggregation (bagging) and N-tuple sampling are explored to propose a retina-free data-driven version of the N-tuple network, whose close analogies to aggregated regression trees, such as classification and regression trees (CART), lead to further architectural enhancements. Performance of the proposed algorithms is compared with the traditional versions of the N-tuple and CART networks on a number of regression problems. The architecture significantly outperforms conventional N-tuple networks while leading to more compact solutions and avoiding certain implementational pitfalls of the latter.

1 Introduction

Recent years have seen a reemergence of interest in the N-tuple network (NTN) architecture (Bledsoe & Browning, 1959). Despite numerous analyses (Bledsoe & Bisson, 1962; Steck, 1962; Roy & Sherman, 1967) and significant progress (Luttrell, 1992; Rohwer, 1995; Kołcz & Allinson, 1996; Jung, Krishnamoorthy, Nagy, & Shapira, 1996; Bradshaw, 1997; Rohwer & Morciniec, 1998), the computational capabilities of this architecture are still not very well understood.

An NTN combines multiple models randomly partitioning the input space. In this article we show that a single NTN node can be represented by a binary tree, which can benefit from learning algorithms similar to those used by classification and regression trees (CART) (Breiman, Friedman, Olshen, & Stone, 1984). This abstraction, together with analogies between N-tuple sampling and bootstrap aggregation (bagging) (Breiman, 1996a), are used to propose a data-driven method of creating a set of NTN tree-based nodes, resulting in a network related to bagged CART.

The article is organized as follows. In section 2 we demonstrate a basic similarity between N-tuple sampling and bagging, and propose a retina-free version of the NTN. Section 3 introduces two tree architectures combining the attributes of CART and NTN. In section 4 numerical experiments are used to demonstrate the advantages of the proposed tree versions of the NTN when compared to the original architecture and to CART on a number of regression problems. Section 5 contains the conclusion.

 © 2000 Massachusetts Institute of Technology

2 Retina-Free NTN and Bagging Estimators

Assuming a binary input pattern space, usually represented in the form of a two-dimensional "retina," NTN performs N-tuple sampling, where a set of K memory nodes, each having an N-bit-long address word, randomly distribute their address lines among the retina bits, with each address line being associated with a particular random feature of the binary input space. The NTN mapping is thus an aggregate of K random low-complexity models.

In continuous variable environments, real-valued inputs are converted into a binary format suitable for N-tuple sampling. In particular, when thermometer encoding (Aleksander & Stonham, 1979) is used, an instance of N-tuple sampling becomes equivalent to placing N quantization thresholds (constrained by a uniform quantization grid in each dimension) within the range of the D input variables (Tattersall, Foster, & Johnston, 1991), leading to hyper-rectangular quantization of the input space. The network is biased by the assumption of uniformity imposed on the input space. The alternative we consider is to bypass the input-conversion stage altogether and perform N-tuple sampling by directly distributing the quantization thresholds within the ranges of the input variables, a process that can be guided by the empirical input-space distribution provided by a training set.

In the light of this idea, certain similarities between the nature of NTN design and some recent advances in learning algorithms become apparent. Breiman (1996a) showed that significant improvements in network prediction performance can be gained by using the concept of bagging, where the variance component of the generalization error is reduced by averaging several variants of the network. Each variant is designed with a different version of the training set, with each version created by sampling with repetitions from the original collection.

To see the similarity between bagging and NTN design, let us first note that each node of an NTN represents a crude model of the desired mapping, which is highly dependent on the positioning of the N-tuple thresholds. Averaging a large number of such randomly designed nodes reduces the variance of the estimate irrespective of the data set considered. However, when N-tuple sampling is constrained to choose the quantization thresholds from within the set training-point coordinates (according to the sampling with repetition regimen), the process of selecting a set of K NTN nodes becomes data driven and effectively corresponds to bagging. Although such an approach should lead to performance improvements, we propose to structure the process of training individual NTN nodes further by taking advantage of their inherent similarity to binary trees.

3 NTN Analogies to CART

CART is a well-known binary tree network based on hyper-rectangular quantization of the input space. In the regression variant, its output to an

input **x** is given by an arithmetic average of the training points falling into the same tree leaf as **x**. During the tree-growing process, which starts with a root node containing all training points, CART chooses the next node to be partitioned such that the split of the corresponding cell maximizes the decrease of the overall training set error. A split is performed parallel to one of the axes by choosing the threshold point out of the set of coordinate values of training points falling into that node. Node splitting continues until a sufficiently large tree is created, at which point a pruning process is initiated to optimize the tree and avoid the problem of overfitting.

Although a node of the NTN is created differently from CART, certain commonalities can be seen. Starting with the unpartitioned input space, the placement of the first quantization threshold of an NTN node is equivalent to splitting the space at the root node of CART. The placement of any additional threshold results in splitting all of the current leaf nodes of the tree at that threshold, instead of splitting exactly one node, as is the case in CART. Despite this important difference, we can create an equivalent tree corresponding to a single node of the NTN, considering that the order of threshold-point placement is fixed (i.e., each N-tuple is ordered). The value of N corresponds to the depth of the equivalent tree.

3.1 Bagged-Tree Implementation of an NTN. In view of the similarities between NTN and CART, a question arises whether the structured approach of tree design could be applied to the NTN in order to enhance the network performance. One obvious problem is that CART constructs essentially one tree, whereas NTN uses K nodes to achieve satisfactory performance levels. However, correspondence between bagging and N-tuple sampling allows substitution of the collection of randomly designed N-tuple nodes for a collection of bagged trees. K bootstrapped CART trees can also be interpreted as nonidentical quantizations of the input space, where each point resides in exactly K quantization cells, as in the NTN. Taking these observations into account, if a structured (i.e., data driven) node design were to be applied to the NTN, bagging could be used to derive the K network nodes.

3.2 Adapting the CART Node-Splitting Criterion. At each step in the CART tree-growing algorithm, all current tree leaves are evaluated for the possibility of a split. The leaf leading to the highest reduction in error is then chosen. The best-split selection process can be expressed as

$$(n, d, t)_{\text{best}} = \arg \max_{(n,d,t)} \left[E(n) - \left(E(n, d, t)_l + E(n, d, t)_r \right) \right], \tag{3.1}$$

where $E(n)$ represents the error estimate at node n, d represents an input variable ($d = 1, \ldots, D$), t is a particular threshold value, and $E(n, d, t)_l$ and $E(n, d, t)_r$ correspond to the error estimates for points at node n residing on both (i.e., "left" and "right") sides of the partitioning threshold $x_d = t$.

Generally, splitting any node at any point will reduce the error (provided that it results in a division of points at that node), and the procedure chooses the best overall split. In the case of the NTN, the splitting occurs for all current terminal nodes. Consequently, it makes sense to choose the best split on the basis of the aggregated decrease of the training-set error, taking all nodes into account, that is,

$$(d,t)_{\text{best}} = \arg\max_{(d,t)} \sum_{n} \left[E(n) - \left(E(n,d,t)_l + E(n,d,t)_r \right) \right]. \tag{3.2}$$

Note that at some nodes, the gain might be zero if all points in that node lie on the same side of the threshold. In such cases when the split is carried out, the tree will have some empty terminal nodes, which might be split again as the process continues. To avoid this undesirable effect, splits are allowed only if the node contains more than one element and the split does not result in an empty cell. In this way no empty cells will be added during the training process.

3.3 Pruning.

3.3.1 Cost-Complexity Pruning. Since every split results in an error reduction, the greedy algorithm outlined above can, in principle, continue until each leaf contains just one element. Clearly, such a network provides a good fit to the training set, but can be expected to have poor generalization properties. To overcome this effect, the CART algorithm utilizes cost-complexity pruning, where the magnitude of the training-set error is weighted against the complexity of the tree measured in terms of the number of its terminal nodes, $E = E_R + \alpha \cdot \widetilde{T}$, where E_R is the mean squared error over the training set and $\widetilde{T}$ is the tree size. After a cross-validated estimate of α is obtained, the tree is pruned by merging pairs of terminal nodes, one pair at a time, until the desired tree complexity is reached. The cost-complexity pruning can be applied directly to the tree created by the NTN algorithm. The resulting tree will not correspond to the NTN anymore, since while for some nodes a particular threshold may be implemented, it may be omitted for others, even though carrying out of the split would not result in the creation of an empty cell. A tree grown according to the NTN strategy and pruned according to the cost-complexity approach will be called CART-NTN. An example input-space quantization characteristic of CART-NTN is illustrated in Figure 1 (left).

3.3.2 Optimal-Depth Pruning. The cost-complexity pruning makes the resulting tree analogous to CART, since the tree can have branches of different lengths not necessarily dictated by the lack of data in the terminal nodes. To keep within the NTN framework, however, each pruning step should merge all cells at the current level. A simple pruning method conforming with the NTN methodology is to choose a depth of the tree that

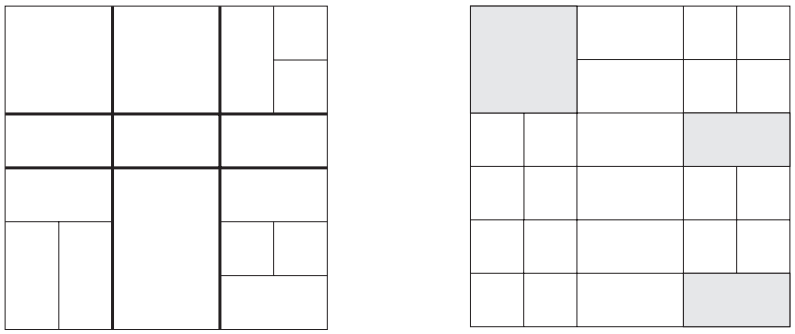


Figure 1: Input space quantization due to (left) CART-NTN and (right) NTN-CART. In CART-NTN, some of the initial quantization thresholds (indicated by heavier lines) will affect the entire input domain, while others will be more local, due to the CART-like pruning procedure. In NTN-CART most of the partitioning thresholds have a global scope, just as in the NTN, but in certain local cases, the splits are not implemented (the shaded regions) due to the lack of training data.

results in the best generalization error, obtained through cross validation. Note that this type of pruning is significantly simpler than cost-complexity pruning. The order of node removal is exactly the same as the order of node generation since we remove all nodes residing at a current level in each stage. Therefore, while the original tree is grown, a cross-validated estimate of the generalization error is computed and associated with that level each time a split is made. After the original tree is grown, the depth corresponding to the minimum cross-validation error is then chosen as the optimal depth of the final tree. This value corresponds essentially to the value of N in the case of NTN. A tree grown according to the NTN strategy and pruned according to the optimal-depth approach will be called NTN-CART. An example input-space quantization typical of NTN-CART is shown in Figure 1 (right).

4 Numerical Experiments

In view of the extensions proposed, CART-NTN and NTN-CART can be considered intermediate architectures, bridging CART with NTN. Performance of CART, NTN, and the intermediate CART-NTN and NTN-CART networks was evaluated in a regression setting using one real and three simulated data sets, which have been used in Breiman (1996a). One hundred random variants of each data set were considered. For the simulated data, the different variants were generated using the formulas given below; for

the real data set, the different samples were obtained by a random split of the whole set into its training and test components. In all experiments with bagged networks 25 bagging repetitions were used. The real-world data set *Boston* corresponds to the *Boston Housing* experiment (Belsley, Kuh, & Welsh, 1980), where a mix of 13 social and environmental variables is used to estimate real estate prices. The total data set consists of 506 elements, where in each random case 481 points were used to train a network and the remaining 25 were used for testing.

The three artificial data sets are due to Friedman (1991). In each random case, a training set consisted of 200 points, and a test set contained 1000 points.

The data set *Friedman-1* is generated according to the formula $y = 10 \cdot \sin(\pi x_1 x_2) + 20 \cdot (x_3 - 0.5)^2 + 10 \cdot x_4 + 5 \cdot x_5 + \varepsilon$, where $\varepsilon \leftarrow N(0, 1)$ and $[x_1, \ldots, x_{10}]$ is a 10-dimensional input vector, whose elements represent independent variables distributed uniformly within $[0, 1]$; ε represents a normally distributed noise component. Note that only 5 of the 10 variables have any influence on the output.

In the case of data sets *Friedman-2* and *Friedman-3*, the variables of the four-dimensional input space are independent and distributed uniformly in the ranges: $x_1 \in [0, 100]$, $x_2 \in [40\pi, 560\pi]$, $x_3 \in [0, 1]$, $x_4 \in [1, 11]$. For the data set *Friedman-2*, the output variable is generated according to $y = \sqrt{x_1^2 + (x_2 x_3 - 1/x_2 x_4)^2} + \varepsilon$ (where $\varepsilon \leftarrow N(0, 127)$), while for the case of the *Friedman-3* data set, the output variable is given by $y = \arctan\left(\frac{x_2 x_3 - 1/x_2 x_4}{x_1}\right) + \varepsilon$ (where $\varepsilon \leftarrow N(0, 0.106)$).

4.1 Network Implementation. CART was implemented according to the description provided in Breiman et al. (1984). The initial tree was grown by successive terminal node splitting (see equation 3.1), while at least one tree leaf contained more than five points and a split reducing the training set error was possible. Cost complexity reduced the number of terminal nodes once the original tree had been grown. Tenfold cross validation was used to estimate the optimum pruning parameter α.

Both CART-NTN and NTN-CART followed a similar initial tree-growing process, except that all current terminal nodes were split at each step, if possible, according to equation 3.2. In the case of CART-NTN, the cost-complexity pruning identical to the one used in CART was then applied, while NTN-CART found the optimal tree depth, minimizing the estimated generalization error obtained via 10-fold cross validation.

In the case of the NTN, in all experiments all inputs were mapped to the $0, \ldots, 255$ range and thermometer encoded to form a binary retina of size $R = 256 \cdot D$, where D is the number of input variables. In one set of experiments, the number K of N-tuple memories was chosen to allow complete sampling of the retina bits for a given value of N, that is, $K \cong R/N$.

Another set of experiments used a generally larger but fixed number of N-tuple nodes ($K = 500$). For each variant of the data set, 25 random N-tuple retina samplings were considered, and the test-set errors obtained in each case were then averaged. The N-tuple memories were implemented using hashing (Knuth, 1973), with each memory node containing 5000 locations. In this way, large N-tuple sizes could be readily considered, and since the training set sizes were in all cases fewer than 500, the probability of hash collisions was negligibly small.

To reduce the generalization error due to empty memory locations, the NTN mapping was modeled as $y = b + NTN(\mathbf{x})$, with the bias b estimated as an average of the training-set values. The network response formation was considered in two variants. One with the output response calculated according to Kołcz and Allinson (1996),

$$NTN(\mathbf{x}) = \frac{\sum_{k=1}^{K} w_k(\mathbf{x})}{\sum_{k=1}^{K} c_k(\mathbf{x})} = \frac{\sum_{t=1}^{T} y^t \cdot \kappa\left(\mathbf{x}, \mathbf{x}^t\right)}{\sum_{t=1}^{T} \kappa\left(\mathbf{x}, \mathbf{x}^t\right)}, \tag{4.1}$$

where T is the size of the training set, $c_k(\mathbf{x})$ is the counter associated with weight $w_k(\mathbf{x})$, $\left(\mathbf{x}^t, y^t\right)$ represents the tth input-output training pair, and $\kappa\left(\mathbf{x}, \mathbf{x}^t\right)$ is the equivalent kernel function of the NTN. In a one-pass training procedure, $w_k(\mathbf{x})$ is set to the sum of training values of points addressing that memory location, and $c_k(\mathbf{x})$ is set to their number. The other form of NTN response was produced according to

$$NTN(\mathbf{x}) = \frac{1}{K} \sum_{k=1}^{K} \frac{w_k(\mathbf{x})}{c_k(\mathbf{x})}, \tag{4.2}$$

with the weights and counters set in the same way. The latter variant directly corresponds to the CART network, since the response of each memory location represents an average of the training-point values that address that location and hence fall into the corresponding quantization cell. Since equation 4.2 makes NTN directly related to the remaining networks, it has been chosen as a basis for performance comparison.

4.2 Discussion of Results. The combined results for all networks and data sets considered are presented in Table 1. The complete results for both variants of the NTN and all values of N considered are shown in Figure 2. The accuracy of CART and CART-NTN was closely matched in all cases (with similar depths of the trees produced), which is not surprising since the same pruning algorithm was applied in both cases. The advantage of one network over the other proved to be data dependent: for the *Friedman-1* and *Friedman-3* experiments, CART-NTN was slightly better than CART, while for the *Friedman-2* and *Boston* experiments, the situation was reversed. It appears that the depths of the trees produced by CART-NTN were unaf-

Table 1: Combined Mean Squared Error (MSE) Results for All Data Sets.

Network	MSE	MSE-bag	Decrease	ATD	ATD-bag
Results for the *Friedman-1* data set					
CART	13.1	9.0	31%	5	7
CART-NTN	12.2	8.7	29%	4	4
NTN-CART	11.4	6.9	40%	4	9
NTN	12.7	12.3	3%	25	25
Results for the *Friedman-2* data set					
CART	36,438	24,840	32%	8	9
CART-NTN	37,930	26,297	31%	4	4
NTN-CART	36,510	24,911	32%	5	14
NTN	32,437	30,797	5%	20	20
Results for the *Friedman-3* data set					
CART	0.0600	0.0370	38%	4	6
CART-NTN	0.0544	0.0346	36%	4	4
NTN-CART	0.0449	0.0281	37%	5	14
NTN	0.045	0.044	2%	20	20
Results for the *Boston* data set					
CART	27.8	16.2	42%	6	7
CART-NTN	31.4	16.2	48%	5	5
NTN-CART	22.6	13.1	42%	8	25
NTN	16.7	15.9	5%	40	45

Notes: MSE-bag refers to the error obtained for the bagged variant of CART, CART-NTN, and NTN-CART and for the variant of NTN with the number of N-tuple nodes fixed to $K = 500$ (in the MSE column, the number of nodes of an NTN is just large enough to cover the retina with N-tuples). A decrease in the test-set error due to bagging is indicated in column 4, and the average tree depths (ATD) (and the value of N for NTN) for the standard and bagged variants of the networks are given in columns 5 and 6, respectively. The NTN results correspond to the optimal values of N.

fected by bagging, whereas for both CART and NTN-CART, bagging led to deeper trees.

NTN-CART proved to be superior to both CART and CART-NTN for all data sets considered, except for the *Friedman-2* experiment, where its performance was essentially equal to that of CART. This can probably be attributed to the optimal-depth pruning used by NTN-CART, which resulted in trees of higher complexity from those used by the other networks. This may indicate that cost-complexity pruning applied to CART and CART-NTN can prune the trees too much in certain cases. Interestingly, the distribution-based trees proposed in Breiman (1996b) also combined higher tree complexity with better prediction accuracy.

All three tree networks demonstrated similar levels of sensitivity to the training data, ranging between 29% and 48% for the data sets considered, where the sensitivity is measured by the decrease of the test-set error due

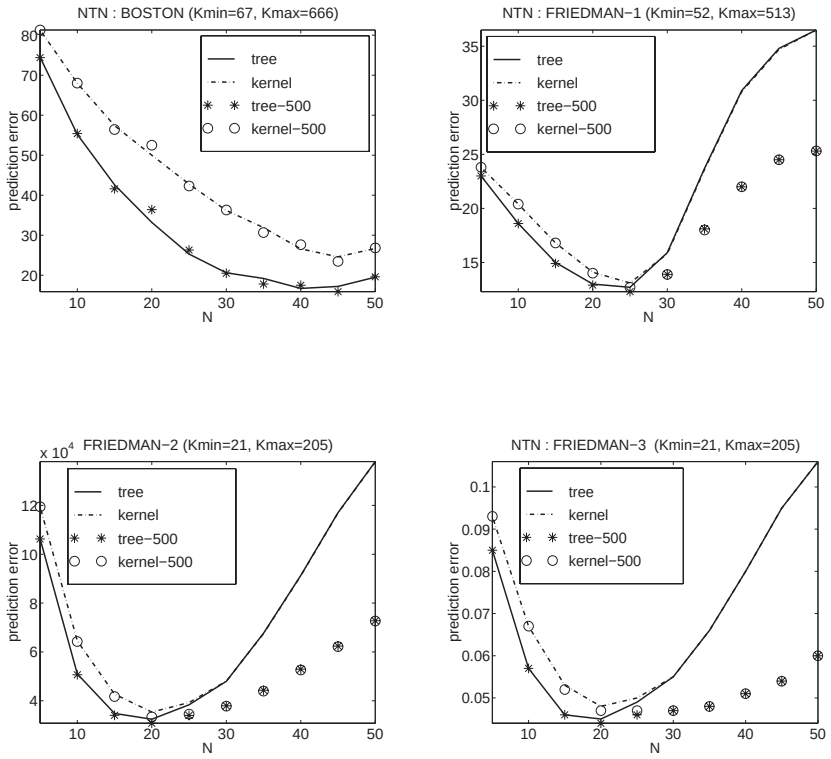


Figure 2: Combined NTN results for the data sets considered and different values of N. The continuous graph lines correspond to NTN with a retina-size dependent K, while the marker graphs refer to the fixed K cases. The *kernel* label refers to the response of an NTN according to equation 4.1 and *tree* according to equation 4.2.

to bagging. This suggests that although CART-NTN and NTN-CART have more data at their disposal than CART when making node-splitting decisions, the splitting process is inherently less stable than in CART since each split affects the whole of the input space.

As far as the N-tuple networks are concerned, their performance proved to be comparable, and in some cases better than the performance of single-node CART, CART-NTN, and NTN-CART, but generally worse than the bagged versions of these networks. This is largely expected and can be attributed to the random nature of N-tuple sampling, which is far from optimal unless the joint distribution of the input-output space is actually uniform. Increasing the resolution of the input retina by using more than 256 quantization levels per input variable did not result in a significant

improvement of NTN performance. Conforming with the results published in Rohwer and Lamb (1993) and Kołcz and Allinson (1996), the optimal values of N tend to be rather large compared to the ones usually used in implementations of the NTN.

Figure 2 confirms that by increasing the number of N-tuple nodes, the network performance is improved. The improvement is often small, which may be attributed to the random nature of N-tuple sampling, which does not take full advantage of the distribution-related information provided by the training data. The performance increase due to large K is more pronounced for the synthetic data sets, for which the input space distribution is truly uniform.

The results obtained for NTN indicate a clear advantage of combining the individual node responses according to equation 4.2 when compared to the regression-NTN method of equation 4.1. The explanation lies in the higher bias induced by equation 4.1. Let $\{w_1, \ldots, w_K\}$ denote the sums of training values in the cells selected for a particular input. Let $\{a_1, \ldots, a_K\}$ and $\{c_1, \ldots, c_K\}$ correspond to the averages and numbers of training points in these cells, respectively. The node aggregation method analogous to bagged CART (see equation 4.2) produces the response as

$$\frac{1}{K}\sum_{k=1}^{K}\frac{w_k}{c_k} = \frac{1}{K}\sum_{k=1}^{K} a_k,$$

while the response of the regression NTN (see equation 4.1) is given by

$$\frac{\sum_{k=1}^{K} w_k}{\sum_{k=1}^{K} c_k} = \frac{\sum_{k=1}^{K} c_k \cdot a_k}{\sum_{k=1}^{K} c_k} = \sum_{k=1}^{K} \beta_k \cdot a_k,$$

where β_k is proportional to the number of training points in the kth cell, and

$$\sum_{k=1}^{K} \beta_k = 1.$$

Thus cell averages generated by larger number of points have more influence on the output in this case. But a larger number of training points falling into a cell is likely to be associated with a larger "footprint" of the cell in the input space, in which many points influencing the cell average a_k are likely to be farther away from the actual input point for which the network response is computed. Consequently, a higher bias will result. For larger values of N, the size of partitioning cells is small enough for the diluting effects of equation 4.1 to become insignificant (see Figure 2). Note that if all cells selected by $\mathbf{x}$ contain an identical number of points, the responses produced according to equations 4.2 and 4.1 will be identical.

5 Conclusion

The analogy between a single N-tuple memory node and a tree network, as well as between N-tuple sampling and bagging, was used to propose a tree network (in two variants: CART-NTN and NTN-CART) using the data-driven, tree-growing strategy of CART and the structure of input-space quantization of an NTN node. The concept of bagging was applied to create a number of such network nodes in a data-driven instead of a purely random way. A simple tree-pruning method maintaining a close similarity to the original NTN architecture has also been suggested. NTN-CART, combining all these features, provides a data-driven, tree-based version of an NTN using the concept of bagging for the generation of multiple network nodes.

Experimental results show that bagged NTN-CART provides an attractive alternative to the NTN, since its data-driven approach to network design is able to match the placement of quantization thresholds to the empirical data distribution provided by the training set. The performance increase is accompanied by a smaller number of nodes, a smaller value of N, and a way of eliminating the problem of empty memory locations characteristic of N-tuple networks. At the same time, the simpler nature of pruning proposed for NTN-CART can also have advantages for this network when compared with the traditional cost-complexity pruning of CART.

References

Aleksander, I., & Stonham, T. J. (1979). A guide to pattern recognition using random-access memories. *IEE Proceedings—E Computers and Digital Techniques, 2*(1), 29–40.

Belsley, D., Kuh, E., & Welsh, R. (1980). *Regression diagnostics*. New York: Wiley.

Bledsoe, W. W., & Bisson, C. L. (1962). Improved memory matrices for the N-tuple pattern recognition method. *IRE Transactions on Electronic Computers, EC-11*, 414–415.

Bledsoe, W., & Browning, I. (1959). Pattern recognition and reading by machine. *IRE Joint Computer Conference*, pp. 225–232.

Bradshaw, N. P. (1997). The effective VC dimension of the N-tuple classifier. In W. Gerstner, A. Germond, M. Hasler, & J. D. Nicoud (Eds.), *Artificial neural networks—ICANN '97. 7th International Conference Proceedings* (pp. 511–516). Berlin: Springer-Verlag.

Breiman, L. (1996a). Bagging predictors. *Machine Learning, 24*(2), 123–140.

Breiman, L. (1996b). Distribution based trees are more accurate. In S. I. Amari, L. Xu, L. W. Chan, I. King, & K. S. Leung (Eds.), *Progress in neural information processing: Proceedings of the International Conference on Neural Information Processing* (pp. 133–138). Berlin: Springer-Verlag.

Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. (1984). *Classification and regression trees*. Monterey, CA: Wadsworth and Brooks/Cole.

Friedman, J. H. (1991). Multivariate adaptive regression splines. *Annals of Statistics, 19*(1), 1–141.

Jung, D. M., Krishnamoorthy, M. S., Nagy, G., & Shapira, A. (1996). N-tuple features for OCR revisited. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 18*(7), 734–745.

Knuth, D. E. (1973). *The art of computer programming* (Vol. 3). Reading, MA: Addison-Wesley.

Kołcz, A., & Allinson, N. M. (1996). N-tuple regression network. *Neural Networks, 9*, 855–869.

Luttrell, S. P. (1992). Gibbs distribution theory of adaptive N-tuple networks. In I. Aleksander & J. Taylor (Eds.), *Artificial neural networks 2: Proceedings of the 1992 International Conference on Artificial Neural Networks* (pp. 313–316). New York: Elsevier.

Rohwer, R. J. (1995). Two Bayesian treatments of the N-tuple recognition method. *Proceedings of the 4th International Conference on Artificial Neural Networks* (pp. 171–176). London: IEE.

Rohwer, R. J., & Lamb, A. (1993). An exploration of the effect of super large N-tuples on single layer RAMnets. In N. M. Allinson (Ed.), *Proceedings of Weightless Neural Network Workshop '93* (pp. 33–37). York, UK: University of York.

Rohwer, R., & Morciniec, M. (1998). The theoretical and experimental status of the N-tuple classifier. *Neural Networks, 11*(1), 1–14.

Roy, R. J., & Sherman, J. (1967). Two viewpoints of K-tuple pattern recognition. *IEEE Transactions on System Science and Cybernetics, 3*(2), 117–120.

Steck, G. P. (1962). Stochastic model for the Bledsoe-Browning pattern recognition scheme. *IRE Transactions on Electronic Computers, 11*, 274–282.

Tattersall, G. D., Foster, S., & Johnston, R. D. (1991). Single-layer lookup perceptrons. *IEE Proc. F, 138*, 46–54.

Received May 1, 1998; accepted February 14, 1999.